

Process Synchronization

Race Condition, Critical Section Problem, Software and Hardware Solutions

Fundamentals of Operating System Design (FOSD)

DEPSTAR - CHARUSAT

Learning Objectives

By the end of this lecture, students should be able to:

- Explain race conditions and why they occur in concurrent programs.
- Identify and analyze the critical section of a process.
- State the three requirements of a correct critical-section solution.
- Explain Peterson's solution for two processes.
- Distinguish busy waiting from blocking synchronization.
- Explain hardware-based synchronization using:
 - disabling interrupts,
 - Compare-and-Swap (CAS),
 - Exchange (XCHG) instruction.
- Compare software and hardware approaches in terms of correctness, portability, and performance.

Why Process Synchronization?

Concurrent processes/threads may access the same shared resource.

Shared resources

- Shared memory
- Global variables
- Files
- Database records
- Kernel data structures
- Device state

Potential problem

- Operations may interleave.
- One process may observe an inconsistent state.
- Final result may depend on timing.
- The same program may produce different results on different executions.

Key idea

Synchronization controls the ordering of concurrent accesses to shared data.

Race Condition

Race condition: A situation where the correctness of a program depends on the relative timing or interleaving of concurrent executions.

Simple example

Suppose two threads execute:

$$\text{counter} = \text{counter} + 1$$

and initially $\text{counter} = 5$.

The statement is typically implemented as:

$$\text{LOAD counter} \rightarrow \text{ADD } 1 \rightarrow \text{STORE counter}$$

Important

The source-level statement looks atomic, but the underlying read-modify-write sequence may not be atomic.

Race Condition: Interleaving Example

Initial value:

$counter = 5$

Step	Thread T1	Thread T2
1	Read $counter = 5$	
2		Read $counter = 5$
3	Compute $5 + 1 = 6$	
4		Compute $5 + 1 = 6$
5	Write 6	
6		Write 6

Expected result after two increments:

$$5 + 1 + 1 = 7$$

Actual result:

$counter = 6$

Race Condition: Real-World Example

Bank account example

Initial balance:

Rs.10,000

Two ATM transactions execute concurrently:

- ATM 1 withdraws Rs. 6,000.
- ATM 2 withdraws Rs. 5,000.

If both transactions read the old balance before either writes the new balance:

$$T1 : 10000 - 6000 = 4000$$

$$T2 : 10000 - 5000 = 5000$$

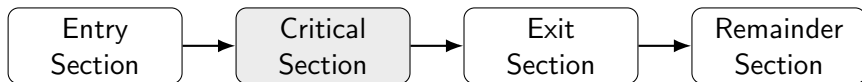
Both writes may succeed even though the account does not have enough funds for both withdrawals.

Lesson

Financial transactions, ticket booking, inventory management, and database updates require synchronization.

Critical Section

Critical section: A segment of code in which a process accesses shared data or a shared resource.



- **Entry section:** request permission to enter.
- **Critical section:** access shared resource.
- **Exit section:** release permission.
- **Remainder section:** other computation.

Critical Section Problem

Consider n concurrent processes:

$$P_0, P_1, \dots, P_{n-1}$$

Each process has:

Entry $\rightarrow$ Critical Section $\rightarrow$ Exit $\rightarrow$ Remainder

Problem

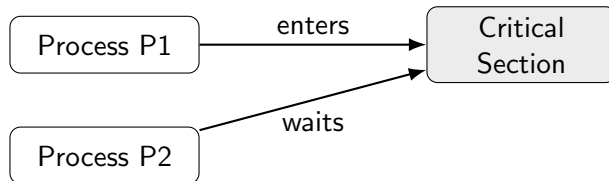
Design a protocol so that concurrent processes can safely access shared data without violating correctness.

A correct solution should satisfy three classical requirements:

- 1 Mutual exclusion
- 2 Progress
- 3 Bounded waiting

Requirement 1: Mutual Exclusion

Mutual exclusion: At most one process can execute inside the critical section at a time.



Violation

If P1 and P2 execute the critical section simultaneously, shared data can be corrupted.

Requirement 2: Progress

If no process is currently in the critical section and some processes want to enter, the selection of the next process cannot be postponed indefinitely.

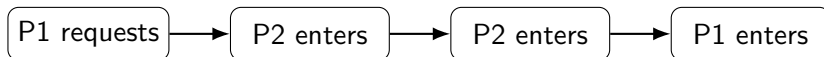
- Only processes that are actually trying to enter should participate in the decision.
- A process in its remainder section should not prevent another process from entering.

Example

If P1 is not interested in the shared resource, P1 should not be able to indefinitely prevent P2 from entering the critical section.

Requirement 3: Bounded Waiting

Bounded waiting: After a process requests entry, there must be a limit on the number of times other processes can enter before it gets its turn.



If P2 can repeatedly enter while P1 waits forever, **starvation** may occur.

Goal

A correct algorithm should prevent indefinite postponement.

Three Requirements at a Glance

Requirement	Meaning
Mutual exclusion	At most one process is in the critical section at any instant.
Progress	If the critical section is free, a waiting process should be allowed to enter without unnecessary delay.
Bounded waiting	A requesting process should not wait indefinitely while others repeatedly enter.

Important distinction

Mutual exclusion prevents simultaneous entry. **Bounded waiting** addresses starvation. **Progress** addresses unnecessary postponement.

Software-only approaches attempt to solve the critical-section problem using shared variables and ordinary read/write operations.

Examples:

- Peterson's solution
- Earlier approaches such as strict alternation
- Software flags and turn variables

Historical significance

These algorithms are important for understanding synchronization principles, even though modern systems normally rely on atomic hardware instructions and OS synchronization primitives.

Peterson's Solution: Basic Idea

Peterson's solution is a classical software solution for **two processes**.

Shared variables:

bool flag[2]; int turn;

Interpretation:

- $flag[i] = true$: process P_i wants to enter.
- $turn = j$: give the other process priority if both want to enter.

For process P_i , let:

$$j = 1 - i$$

The key idea combines:

interest flag + turn variable

Peterson's Solution: Pseudocode

```
do {  
    flag[i] = true;  
    turn = j;  
  
    while (flag[j] && turn == j)  
        ;    // busy waiting  
  
    /* critical section */  
    flag[i] = false;  
    /* remainder section */  
} while (true);
```

For P_0 : $j = 1$.

For P_1 : $j = 0$.

Observation

A process announces interest and then gives the other process the opportunity to proceed first.

Peterson's Solution: Why Mutual Exclusion Works

Suppose both processes want to enter:

$$flag[0] = flag[1] = true$$

Only one value can remain in:

$$turn$$

Suppose:

$$turn = 1$$

Then:

$$P_0 : flag[1] \wedge (turn = 1) \Rightarrow \text{wait}$$

while:

$$P_1 : flag[0] \wedge (turn = 0) \Rightarrow \text{false}$$

Therefore P_1 can enter the critical section.

Key idea

When both compete, the final value of `turn` determines which process gets priority.

Peterson's Solution: Properties

Under the classical sequentially consistent model, Peterson's algorithm satisfies:

- **Mutual exclusion**
- **Progress**
- **Bounded waiting**

Why it is important

Peterson's solution demonstrates that synchronization can be constructed conceptually using only shared-memory reads and writes.

Modern systems

On modern multicore processors, compiler and CPU memory reordering can invalidate the naive implementation unless appropriate memory-ordering mechanisms are used.

Peterson's Solution: Limitation

- Designed for only two processes.
- Uses busy waiting.
- Consumes CPU cycles while waiting.
- Sensitive to memory-ordering assumptions on modern hardware.
- Not normally used as the production synchronization primitive in modern OS kernels/applications.

Educational value

Peterson's algorithm is still extremely useful for understanding mutual exclusion, progress, bounded waiting, and the role of atomicity.

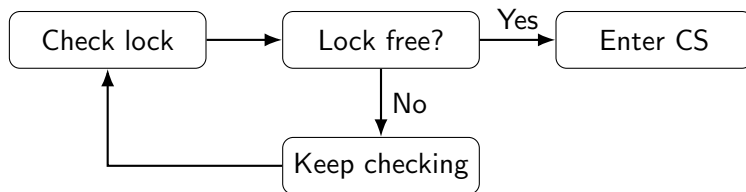
Busy Waiting

Busy waiting: A process repeatedly checks a condition while waiting for a resource.

Typical form:

```
while(lock == 1);
```

The CPU executes the loop even though useful work is not being performed.



Busy Waiting: When Can It Be Useful?

Busy waiting is not always wrong.

- Useful when the expected wait is extremely short.
- Can avoid the overhead of putting a thread to sleep and waking it.
- Common in low-level synchronization and spinlocks.

Trade-off

Short wait $\Rightarrow$ spinning may be acceptable

Long wait $\Rightarrow$ blocking is usually preferable

Example: A kernel spinlock may be appropriate when the protected critical section takes only a few hundred instructions.

Busy Waiting vs Blocking

	Busy Waiting	Blocking
CPU while waiting	Continues executing checks	Can run another thread
Context switch	Usually avoided	Usually involved
Short wait	Can be efficient	May have unnecessary overhead
Long wait	Wastes CPU cycles	Usually more efficient
Typical primitive	Spinlock	Mutex/semaphore

Design principle

Choose spinning or blocking according to expected critical-section duration and scheduling context.

Why Hardware Support Is Needed

Software algorithms rely on operations such as:

read and write

But two separate operations can be interleaved.

Hardware can provide an operation that is **atomic**:

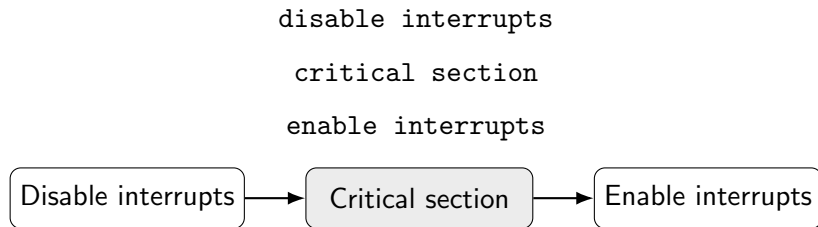
indivisible with respect to competing processors

Important hardware techniques:

- 1 Disable interrupts
- 2 Compare-and-Swap (CAS)
- 3 Exchange / XCHG

Hardware Solution 1: Disable Interrupts

On a single CPU, a kernel can temporarily disable interrupts while executing a critical section. Conceptually:



If the CPU cannot be interrupted, another process cannot be scheduled on that CPU during the critical section.

Disable Interrupts: Advantages and Problems

Advantages

- Simple concept.
- Effective for short kernel critical sections on a uniprocessor.
- Prevents interrupt-driven preemption on that CPU.

Problems

- User programs must not be allowed to disable interrupts.
- Long critical sections can increase interrupt latency.
- On multiprocessors, disabling interrupts on one CPU does not stop another CPU.
- Therefore it is insufficient as a general multiprocessor locking mechanism.

Key point

Disabling interrupts is primarily a low-level kernel technique, not a general-purpose application synchronization method.

Hardware Solution 2: Compare-and-Swap (CAS)

Compare-and-Swap atomically performs:

if $*address == expected$ then $*address = new$

The operation returns the old value.

Conceptual form:

$CAS(address, expected, new)$

The comparison and update occur as one atomic hardware operation.

Lock acquisition

If:

$lock = 0$

then:

$CAS(\&lock, 0, 1)$

changes the lock to 1 atomically.

CAS-Based Spinlock

```
while (compare_and_swap(&lock, 0, 1) != 0)
    ;           // busy waiting

/* critical section */

lock = 0;
```

Interpretation:

- $lock = 0$: unlocked.
- $lock = 1$: locked.
- First successful CAS changes $0 \rightarrow 1$.
- Other processors continue spinning until the lock is released.

Why it works

Multiple processors may execute CAS concurrently, but only one can atomically change the lock from 0 to 1.

CAS: Step-by-Step Example

Initial:

$$lock = 0$$

Two processors P1 and P2 simultaneously execute:

$$CAS(lock, 0, 1)$$

Processor	Atomic operation	Result
P1	$CAS(0, 0, 1)$	succeeds; lock becomes 1
P2	$CAS(0, 0, 1)$	fails because lock is now 1

Therefore:

Only P1 enters the critical section

After P1 finishes:

$$lock = 0$$

P2 can retry.

Hardware Solution 3: Exchange Instruction

The Exchange (XCHG) instruction atomically swaps two values.
Conceptually:

$$XCHG(register, memory)$$

A common lock implementation:

$$key = 1$$

A process repeatedly exchanges a local variable with the lock:

$$XCHG(lock, key)$$

If the returned lock value is 0, the process acquired the lock.

Atomicity

The swap itself is performed atomically by the processor, preventing two processors from simultaneously claiming the same lock.

Exchange-Based Lock

```
key = 1;  
  
do {  
    exchange(&lock, &key);  
} while (key != 0);  
  
/* critical section */  
  
lock = 0;
```

Conceptual state:

<i>lock</i>	<i>Meaning</i>
0	<i>unlocked</i>
1	<i>locked</i>

If a process exchanges $lock = 0$ with $key = 1$, it receives 0 and therefore acquires the lock.

CAS vs Exchange

	Compare-and-Swap	Exchange
Basic action	Compare and conditionally update	Atomically swap values
Conditional?	Yes	No
Typical use	Locks, lock-free algorithms	Simple spinlocks
Typical abstraction	CAS(address, old, new)	XCHG(register, memory)
Failure behavior	Operation can report failure	Exchange always swaps

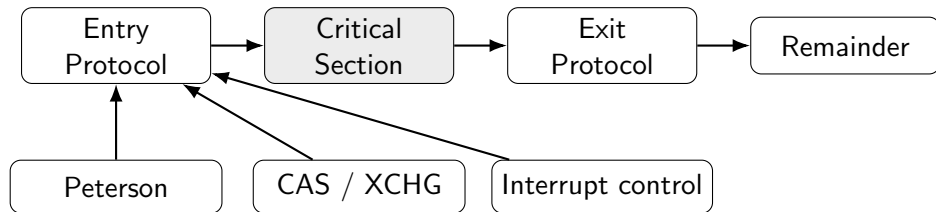
Common idea

Both provide an atomic primitive that cannot be safely constructed from ordinary non-atomic reads and writes alone.

Software vs Hardware Synchronization

Aspect	Software	Hardware
Example	Peterson	CAS, XCHG
Atomic primitive	Ordinary reads/writes	CPU atomic instruction
Generality	Mainly educational/classical	Widely used in systems
Multicore support	Requires careful memory ordering	Designed for shared-memory concurrency
CPU waiting	Peterson uses busy waiting	Spinlocks often busy-wait
Modern use	Limited	Fundamental building block

Critical Section: Complete Design View



The synchronization mechanism must ensure:

Mutual Exclusion + Progress + Bounded Waiting

where the applicable guarantees depend on the mechanism and implementation.

Worked Analysis: Counter Update

Two threads each execute shared counter variable:

$$\textit{counter} = \textit{counter} + 1$$

Initial:

$$\textit{counter} = 100$$

Without synchronization:

possible final value = 101

With correct mutual exclusion:

final value = 102

Reason

Each read–modify–write operation completes before another thread can modify the counter.

Worked Analysis: Counter Update

Possible solutions:

- Peterson's solution for the classical two-process case.
- CAS-based lock.
- XCHG-based spinlock.
- Higher-level mutex/semaphore provided by the OS.

Design Trade-offs

Peterson

- Software only
- Two processes
- Busy waiting
- Excellent conceptual model

Disable Interrupts

- Kernel-level
- Useful on one CPU
- Very short critical sections
- Not sufficient alone for multiprocessors

CAS / XCHG

- Hardware atomicity
- Suitable for multiprocessor systems
- Foundation for locks
- Spin waiting may consume CPU

System-level perspective

Modern synchronization mechanisms generally build higher-level abstractions such as mutexes, semaphores, monitors, and atomic data structures on top of hardware-supported atomic operations.

Common Mistakes

- ❶ Assuming a source-level statement is automatically atomic.
- ❷ Confusing mutual exclusion with bounded waiting.
- ❸ Treating busy waiting as always incorrect.
- ❹ Using Peterson's algorithm without considering modern memory ordering.
- ❺ Assuming disabling interrupts solves synchronization across all CPUs.
- ❻ Forgetting that CAS/XCHG provide atomic primitives, not automatically a complete fair locking policy.
- ❼ Ignoring starvation and performance when designing synchronization.

GATE / Exam-Oriented Question 1

Two processes execute:

$$x = x + 1$$

concurrently, where x is initially 10.

Which final values can occur if the read–modify–write sequence is not atomic?

- ① Only 12
- ② Only 11
- ③ 11 or 12
- ④ 10 or 12

Answer

11 or 12

If both processes read 10 before either writes, the final value can be 11. If their operations do not overlap, it becomes 12.

A critical-section solution allows multiple processes to enter simultaneously, but guarantees that a requesting process will eventually enter.

Which property is violated?

- ① Progress only
- ② Bounded waiting only
- ③ Mutual exclusion
- ④ None

Answer

Mutual exclusion

The defining requirement is that at most one process executes the critical section at a time.

HOTS Question: Compare Two Designs

A multicore application protects a shared counter.

Design A:

```
while(lock is busy) do nothing
```

Design B: The waiting thread is blocked and another runnable thread is scheduled.

Analyze:

- ① Which design is preferable for a 50-nanosecond critical section?
- ② Which design is preferable if the lock may be held for several milliseconds?
- ③ What workload characteristics affect the decision?

Expected reasoning

Discuss context-switch overhead, CPU utilization, contention, critical-section duration, and scheduling behavior rather than memorizing a single rule.

HOTS Question: CAS Reasoning

Suppose:

$$lock = 0$$

P1 and P2 simultaneously execute:

$$CAS(lock, 0, 1)$$

- ① Explain why both cannot successfully acquire the lock.
- ② What value does the unsuccessful process observe?
- ③ What happens after the successful process executes $lock = 0$?
- ④ What problem can arise if the unsuccessful process spins for a very long time?

Key concepts

Atomicity, mutual exclusion, retry loop, busy waiting, and CPU consumption.

Real-World Analogy: Single Bathroom

Consider a bathroom with one key.

- **Key available** → resource is free.
- **Person takes key** → enters critical section.
- **Others wait** → mutual exclusion.
- **Person returns key** → exit section.

Different synchronization strategies can be imagined:

- **Busy waiting:** repeatedly check whether the key is available.
- **Blocking:** wait elsewhere until notified.
- **CAS/XCHG:** atomically claim the key.

Conceptual connection

The shared resource is the bathroom; the lock/key represents the synchronization mechanism.

Summary

- A **race condition** occurs when correctness depends on concurrent execution timing.
- A **critical section** accesses shared data/resource.
- A correct critical-section solution requires:

Mutual Exclusion + Progress + Bounded Waiting

- **Peterson's solution** is a classical two-process software solution.
- **Busy waiting** repeatedly checks a condition and can be useful for very short waits.
- **Disabling interrupts** is a low-level kernel technique, especially on uniprocessors.
- **CAS** conditionally updates a value atomically.
- **Exchange/XCHG** atomically swaps values and can implement locks.
- Modern OS synchronization primitives commonly rely on hardware atomic instructions.

Concept Map

